Tree(Unit-6)

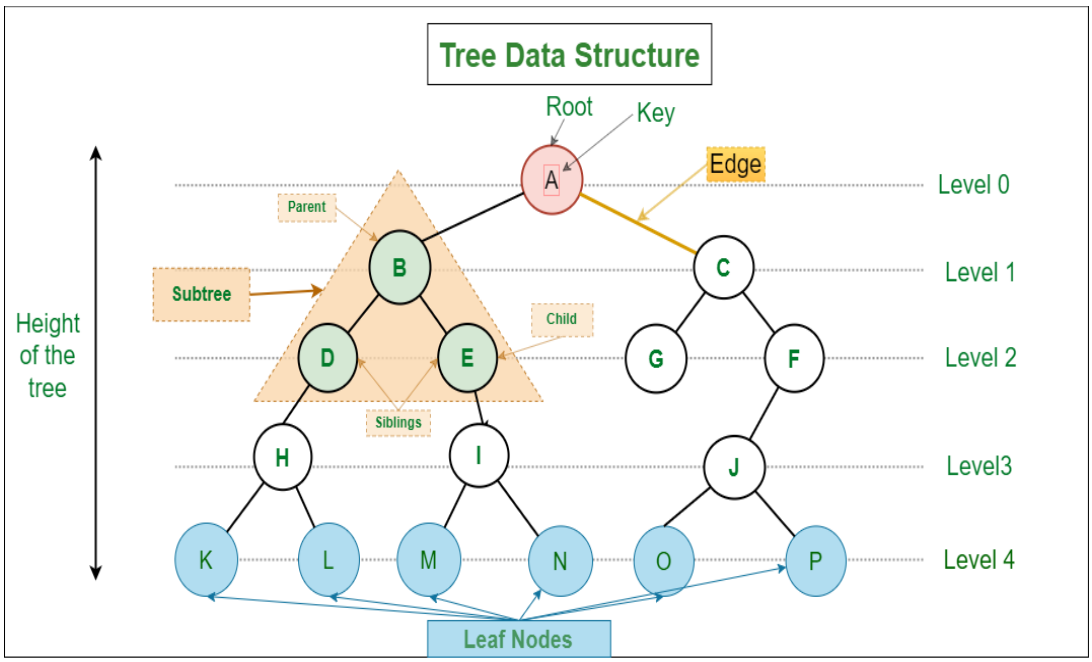
A tree is a hierarchical data structure that consists of nodes connected by edges. It is a nonlinear data structure that represents a set of elements in a hierarchical manner. A hierarchical manner is a way of organizing things in a system of levels. The highest level is at the top, and the lowest level is at the bottom.

In a tree, there is a special node called the root node that serves as the starting point of the tree. The root node can have zero or more child nodes, each of which may have its own child nodes, forming a branching structure. The nodes in a tree are typically connected in a parent-child relationship, where each child node has a single parent node. Nodes are organized at different levels. Node of level 'l' is the parent of node at level 'l+1'. Tree does not contain a cycle, that is, it is not possible to come back to the node from where traversal or searching was started.

Trees are ADT (Abstract Data Structures) which form a hierarchical layout comprising of a single root node followed by parent nodes and children's nodes, which are connected to each other via edges. Since trees are flexible and powerful data structures, they are multipurpose data structure that provide a wide range of applications to the user.

Applications of Tree:

- Storing hierarchical data: Trees are a natural way to store data that is organized in a hierarchical structure, such as the file system on a computer or the organization chart of a company.
- Searching: Trees can be used to search for data quickly and efficiently. For example, a binary search tree can be used to search for a specific value in a sorted list of elements in logarithmic time.
- Traversal: Trees can be traversed in a variety of ways, depending on the application. For example, a depth-first traversal can be used to print all of the nodes in a tree, while a breadth-first traversal can be used to find the shortest path between two nodes.
- Sorting: Trees can be used to sort data. For example, a binary search tree can be used to sort a list of elements in ascending or descending order.
- Compression: Trees can be used to compress data. For example, a Huffman tree can be used to compress a text file by representing common characters with shorter codes.
- Decision making: Trees can be used to make decisions. For example, a decision tree can be used to help a doctor diagnose a patient by considering a series of symptoms.



Terminologies used in Tree :

- **Node:** Each element in a tree is called a node. It represents a single entity in the tree and can hold data.
- **Root:** The root is the topmost node of a tree. It is the starting point or the main access point to the tree.
- **Parent:** The parent of a node is the node directly above it. It is the immediate ancestor of the node.
- **Child:** The children of a node are the nodes directly below it. They are the immediate descendants of the node.
- **Siblings:** Nodes that share the same parent are called siblings. They are at the same level in the tree.
- **Leaf:** A leaf node, also known as a terminal node, is a node that has no children. It is at the end of a branch.
- **Internal Node:** An internal node is any node in the tree that is not a leaf. It has at least one child.
- **Edge:** An edge is a connection between two nodes. In the picture, the edges are the lines that connect the nodes.
- **Path:** A path is a sequence of nodes that connects two nodes in a tree. It starts from one node and ends at another.
- **Depth:** The depth of a node is the number of edges from the root to that node.
- **Height:** The height of a tree is the maximum depth among all the nodes in the tree.

- **Subtree:** A subtree is a portion of the tree that consists of a node and all its descendants.
- **Degree:** The degree of a node is the number of children it has.
- **Binary Tree:** A binary tree is a tree in which each node has at most two children, referred to as the left child and the right child.
- **Balanced Tree:** A balanced tree is a tree in which the heights of the left and right subtrees of every node differ by at most one.

Game tree:-

A game tree isn't a special new data structure—it's a name for any regular tree that maps how a discrete game is played. Lets take an example of game called Rocks. In this game, we have different piles of rocks, with one or more rocks in each pile. The game has two players, who take turns taking one or more rocks from a single pile until one pile is left. When one pile is left, our goal is to force our opponent to remove the last rock. The person who removes the last rock loses.

Figure 48: Simple game of rock with 2 piles

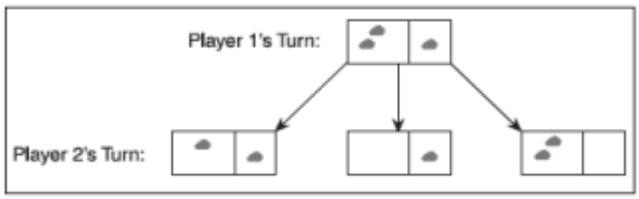


Figure 49: First 2 levels of game tree

In the figure, there are two piles. The first pile has two rocks, and the second pile has only one rock. The first player has three choices:

• Remove one rock from pile 1.

• Remove two rocks from pile 1.

• Remove one rock from pile 2.

We can start the game off with one of those three moves. We can create a simple game tree to represent these moves, as shown in Fig25. After Player 1 has moved, it is now Player 2's turn. Player2's choice of a move is limited to the current state of the game, however. In the leftmost state of Fig24, Player 2 has two choices: He can remove one rock from pile 1 or one rock from pile 2. His choice for the middle state is even less useful He can only remove one rock from pile 2. Of course, because this is the last rock, Player 2 has lost the game. On the right state, Player 2 has two options again: He can remove one or two rocks from pile 2. Fig26 shows the game tree for all five of these moves and goes down one more level to show the complete game tree.

The game is entirely complete by the time the fourth level is reached. The game can have up to three moves because there were only three rocks. We can also tell from the tree that there are five total outcomes from the game because there are five leaf nodes. The game always ends on a leaf node because there are no more moves that can be made. So what can we tell about the game tree that we couldn't easily tell about the initial game setup? For Player1, the obvious first move

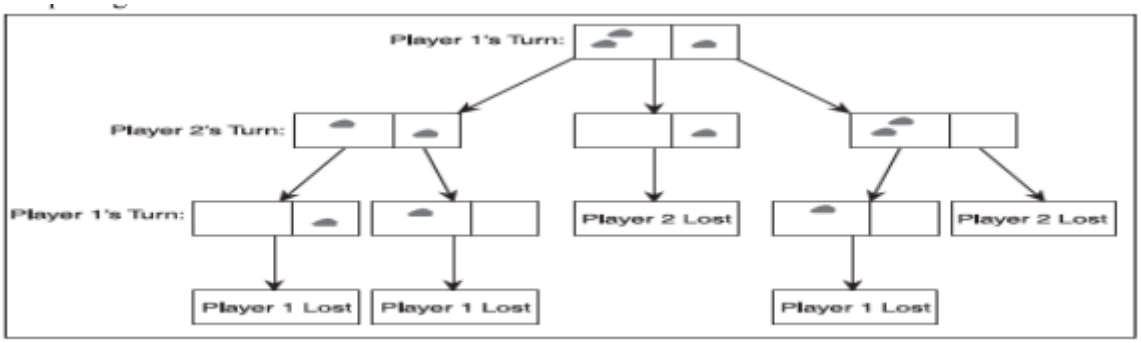
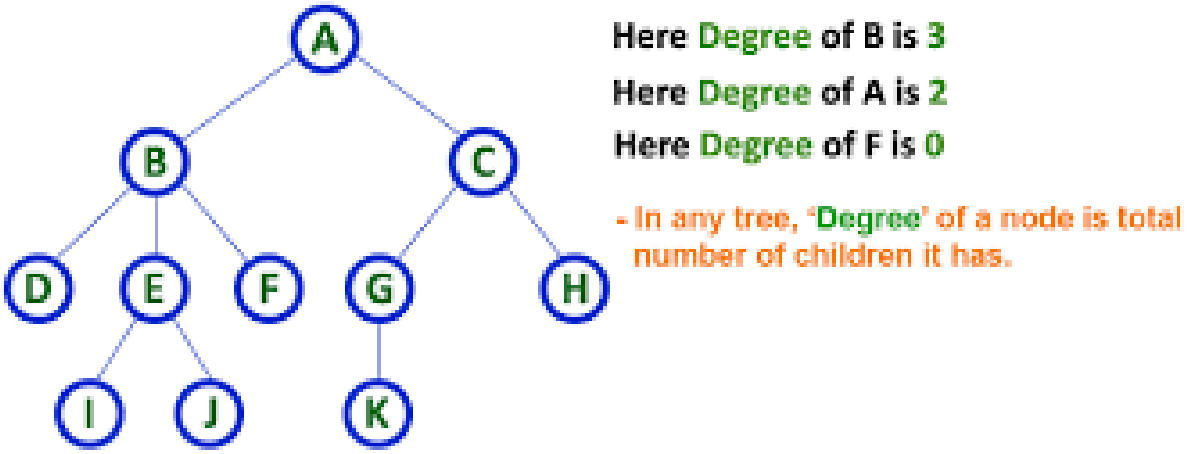


Figure 50: A Complete Game tree

is the second one, removing the two stones from pile 1. By doing that, Player 2 forced to be lost the game, because he has no other option and cannot possibly win. Another thing that we would notice if Player 1 plays the leftmost move, removing one rock from pile 1, is a death sentence. If Player 1 makes that move, then he have given Player 2 a free win, because no matter what move he makes, there is no chance for Player 1 to win in that branch. If Player 1 takes the third route on the opening move, then Player 2 decides the outcome of the game. If Player 2 removes both rocks in pile 1 (a very stupid move), he loses. If he only removes one rock, then he forces Player 1 to remove the last one, and he loses.

Binary Tree: -

A binary tree is a tree data structure in which each node can have at most two children. Sometimes referred to as the left child and the right child. In a binary tree, each node can have a maximum degree of 2. A node with no children is called a leaf node or a terminal node. It has a degree of 0. A node with one child has a degree of 1. The child can be either the left child or the right child. A node with two children has a degree of 2. It has both a left child and a right child.



Binary Tree Representations

A binary tree data structure is represented using two methods. Those methods are as follows...

1. Array Representation
    - In array representation of a binary tree, we use one-dimensional array (1-D Array) to represent a binary tree.

6.3.1 ARRAY REPRESENTATION

An array can be used to store the nodes of a binary tree. The nodes stored in an array of memory can be accessed sequentially. Suppose a binary tree T of depth d. Then at most 2^d-1 nodes can be there in T. (i.e.,SIZE = 2^d–1).Consider a binary tree in Fig8 of depth 3. Then SIZE = $2^3 - 1 = 7$. Then the array A [7] is declared to hold the nodes.

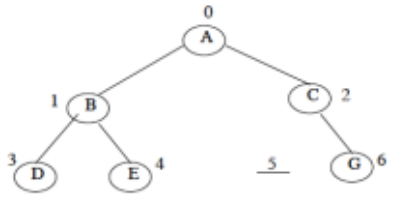


Figure 22: Binary Tree of Depth 3

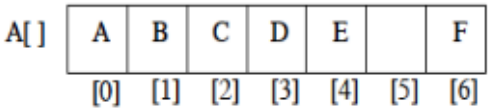


Figure 23: Array Representation of Binary Tree of Fig26

To perform any operation on Binary tree we have to identify the father, the left child and right child of node.

1. The father of a node having index **n** can be obtained by **(n – 1)/2**. For example to find the father of D, where array index n = 3. Then the father nodes index can be obtained
   = (n – 1)/2

   = 3 – 1/2
   = 2/2
   = 1
   i.e., A[1] is the father D, which is B.
2. The left child of a node having index n can be obtained by (2n+1). For example to find the left child of C, where array index n = 2. Then it can be obtained by
   = (2n +1)
   = 2*2 + 1
   = 4 + 1
   = 5
   i.e., A[5] is the left child of C, which is NULL. So no left child for C.
3. The right child of a node having array index n can be obtained by the formula (2n+ 2). For example to find the right child of B, where the array index n = 1. Then
   = (2n + 2)
   = 2*1 + 2
   = 4
   i.e., A[4] is the right child of B, which is E.
4. If the left child is at array index n, then its right brother is at (n + 1). Similarly, if the right child is at index n, then its left brother is at (n – 1).

The array representation is more ideal for the complete binary tree. The Fig8 is not a complete binary tree. Since there is no left child for node C, i.e., A[5] is vacant. Even though memory is allocated for A [5] it is not used, so wasted unnecessarily.

2. Linked List Representation
    - We use a double linked list to represent a binary tree. In a double linked list, every node consists of three fields. First field for storing left child address, second for storing actual data and third for storing right child address.

6.3.2 LINKED LIST REPRESENTATION

The most popular and practical way of representing a binary tree is using linked list (or pointers). In linked list, every element is represented as nodes. A node consists of three fields such as:

(a) Left Child (LChild)
(b) Information of the Node (Info)
(c) Right Child (RChild)

The LChild links to the left child of the parent's node, info holds the information of every node and RChild holds the address of right child node of the parent node. The following figures shows that the Linked List representation of the binary tree of Fig8.

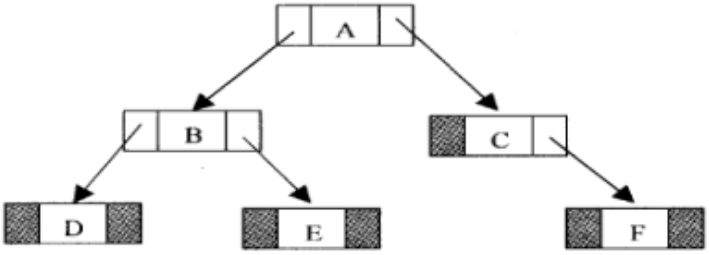


Figure 24: Linked List representations of Binary Tree

If a node has no left or/and right node. Corresponding LChild and RChild is assigned to NULL. The node structure in C can be represented as:

```
struct Node {
      int info;
      struct Node *LChild;
      struct Node *RChild;
};
```

Types of Binary Tree

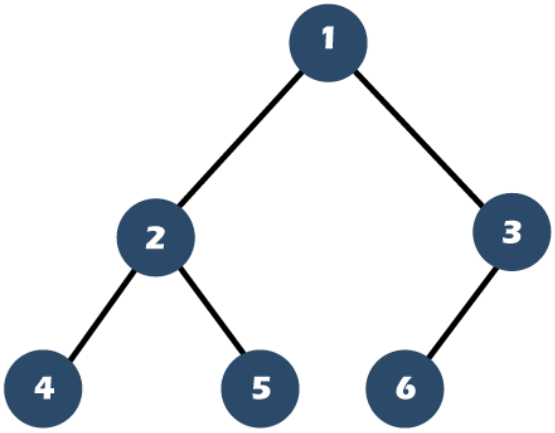
a) Full Binary Tree/Proper/

- A binary tree is said to be a Full binary tree if all nodes except the leaf nodes have either 0 or 2 children. In other words the degree of such tree can either be 0 or 2.

- In the given image, we can see that, nodes 'a', 'c', 'd' have two child nodes each. nodes 'b', 'e' have 0 child nodes.

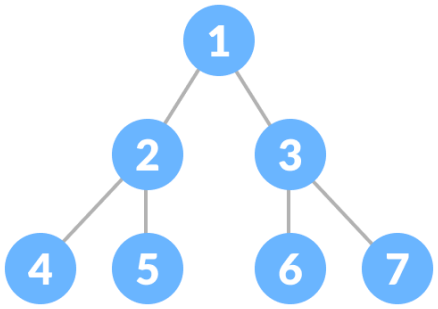
b) Complete Binary Tree

- A complete binary tree is a binary tree in which all levels except possibly the last level are completely filled, and all nodes are left-justified.
- In a complete binary tree, the last level is filled from left to right.
- At the lowest level the nodes have (by definition) zero children, and at the level above that nodes can have 0, 1 or 2 children. An example is given in the following figure.



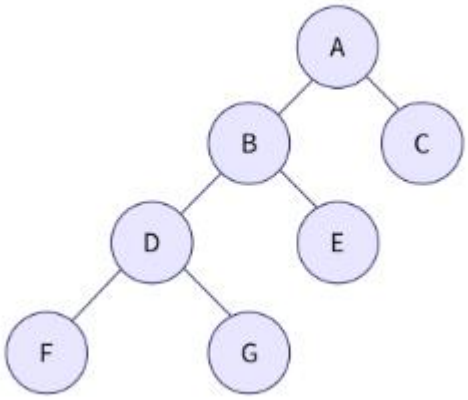
c) Perfect Binary Tree

- A perfect binary tree is a binary tree in which all internal nodes have two children.
- all leaf nodes are at the same level.
- In other words, all levels of the tree are completely filled.
- Every leaf node in a perfect binary tree must be present at the same level. A perfect binary tree is also called a strict binary tree.



d) Almost Complete Binary Tree

- All levels except possibly the last level are completely filled.
- All nodes of the last level are as far left as possible.
- A special kind of binary tree where insertion takes place level by level and from the left at each level. The last level may be partially or fully filled.



e) Skewed binary tree

- It is a type of binary tree in which all the nodes are either left-skewed or right-skewed. In other words, all the nodes in the tree have only one child, either on the left side or the right side. In a left skewed tree, most of the nodes have the left child without corresponding right child.
- In a right skewed tree, most of the nodes have the right child without corresponding left child
- In a left skewed tree, most of the nodes have the left child without corresponding right child.
- In a right skewed tree, most of the nodes have the right child without corresponding left child

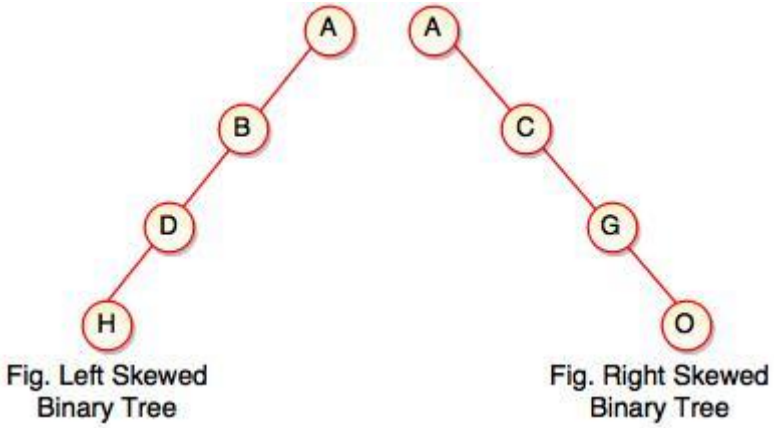
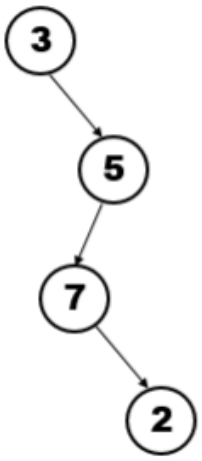


Fig. Left Skewed Binary Tree

Fig. Right Skewed Binary Tree

f) Degenerate binary tree

- A degenerate binary tree, also known as a pathological tree, is a type of binary tree where each parent node has only one child. In other words, it is a tree that is heavily skewed to one side, either left or right



G) Strict Binary Tree

- Only if each node has either 0 or 2 offspring can the tree be regarded as strict binary tree. The term "strict binary tree" can also refer to a tree where all nodes, save the leaf nodes, must have two children.The maximum number of nodes is 2h+1 -1, which corresponds to the number of nodes in the binary tree.
- The whole binary tree must have at least 2*h-1 nodes.
- The whole binary tree has a minimum height of log2(n+1) - 1.

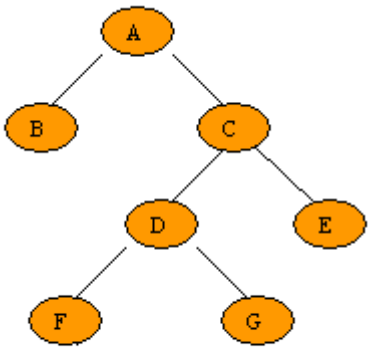


Fig .5.1.4 Strictly binary tree

6.5 TRAVERSING BINARY TREE

Tree traversal is one of the most common operations performed on tree data structures. It is a way in which each node in the tree is visited exactly once in a systematic manner. There are three standard ways of traversing a binary tree. They are:

1. Pre Order Traversal (Node-left-right)
2. In order Traversal (Left-node-right)
3. Post Order Traversal (Left-right-node)

6.5.1 Pre Order Traversal (Root-Left-Right)

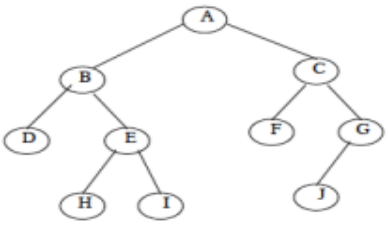
A systematic way of visiting all nodes in a binary tress that visits a node, then visits the nodes in the left sub tree of the node, and then visits the nodes in the right sub tree of the node.

1. Visit the root node
2. Traverse the left sub tree in preorder
3. Traverse the right sub tree in preorder

That is in preorder traversal, the root node is visited (or processed) first, before traveling through left and right sub trees recursively. C/C++ implementation of preorder traversal technique is as follows:

```
void preorder (Node * Root)
{
      If (Root != NULL)
      {
            printf ("%d\n",Root → Info);
            preorder(Root → Lchild);
            preorder(Root → Rchild);
      }
}
```

The preorder traversal of the following Binary tree is: A,B,D,E,H,I,C,F,G,J



6.5.2 In Order Traversal (Left-Root-Right)

A systematic way of visiting all nodes in a binary tress that visits the nodes in the left sub tree of a node, then visits the node, and then visits the nodes in the right sub tree of the node.

1. Traverse the left sub tree in order
2. Visit the root node
3. Traverse the right sub tree in order

In order traversal, the left sub tree is traversed recursively, before visiting the root. After visiting the root the right sub tree is traversed recursively. C/C++ implementation of inorder traversal technique is as follows:

```
void inorder (NODE *Root)
{
      If (Root != NULL){
            inorder(Root → Lchild);
            printf ("%d\n",Root → info);
            inorder(Root → Rchild);
      }
}
```

The in order traversal of a binary tree in Fig10 is: D, B, H, E, I, A, F, C, J, G.

6.5.3 Post Order Traversal (Left-Right-Root)

A systematic way of visiting all nodes in a binary tress that visits the node in the left sub tree of the node, then visits the node in the right sub tree in the node, and then visits the node.

1. Traverse the left sub tree in post order
2. Traverse the right sub tree in post order
3. Visit the root node

In Post Order traversal, the left and right sub tree(s) are recursively processed before visiting the root. C/C++ implementation of post order traversal technique is as follows:

```
void postorder (NODE *Root)
{
      If (Root != NULL)
      {
            postorder(Root → Lchild);
            postorder(Root → Rchild);
            printf ("%d\n", Root → info);
      }
}
```

The post order traversal of a binary tree in Fig10 is: D, H, I, E, B, F, J, G, C, A.

Binary Search Tree

It is called a binary tree because each tree node has a maximum of two children. A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node.

Following are the properties of Binary Search Tree (BST);

i) All nodes of left sub tree are lesser.

ii) All nodes of right sub tree are greater.

iii) Left and right sub tree are should also follow BST properties.

iv) There are no duplicate nodes.

V) In-order traversal of a BST gives an ascending sorted array

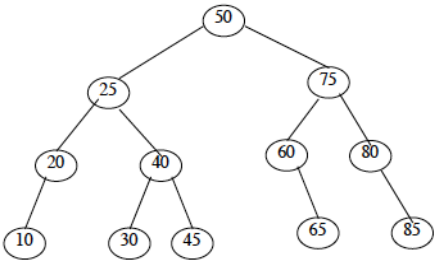


Figure 26: Binary Search Tree

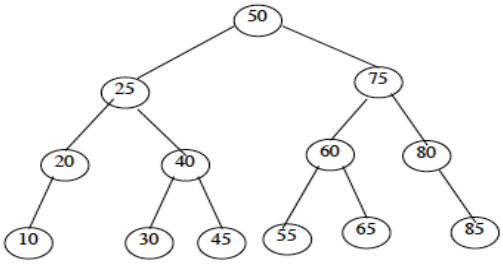


Figure 27: 55 is Inserted in BST of Fig30

All the primitives operation performed on Binary tree can also performed in Binary Search Tree (BST).

6.6.1 Inserting A Node In Binary Search Tree

A BST is constructed by the repeated insertion of new nodes to the tree structure. Inserting a node in to a tree is achieved by performing two separate operations.

1. The tree must be searched to determine where the node is to be inserted.
2. Then the node is inserted into the tree.

Suppose a "DATA" is the information to be inserted in a BST.

Step 1: Compare DATA with root node information of the tree
(i) If (DATA < ROOT → Info)
Proceed to the left child of ROOT
(ii) If (DATA > ROOT → Info)
Proceed to the right child of ROOT
Step 2: Repeat the Step 1 until we meet an empty sub tree, where we can insert the DATA in place of the empty sub tree by creating a new node.
Step 3: Exit

For example, consider a binary search tree in Fig11. Suppose we want to insert a DATA = 55 in to the tree, then following steps one obtained:

1. Compare 55 with root node info (i.e., 50) since 55 > 50 proceed to the right sub tree of 50.
2. The root node of the right sub tree contains 75. Compare 55 with 75. Since 55 < 75 proceed to
the left sub tree of 75.
3. The root node of the left sub tree contains 60. Compare 55 with 60. Since 55 < 60 proceed to the right sub tree of 60.
4. Since left sub tree is NULL place 55 as the left child of 60 as shown in above Fig12.

6.6.2 Algorithm For Inserting An Element Into BST

NEWNODE is a pointer variable to hold the address of the newly created node. DATA is the information to be pushed and TEMP is the Temporary pointer variable of type NODE

1. Input the DATA to be pushed and ROOT node of the tree.
2. NEWNODE = Create a New Node.
3. If (ROOT == NULL)
ROOT=NEWNODE
4. ELSE
    i. TEMP = ROOT
    ii. Repeat until true (while (1))
        a. IF (DATA < TEMP →info)
            i. IF (TEMP →LChild not equal to NULL)
                TEMP = TEMP→LChild
            ii. ELSE

TEMP→LChild = NewNode
Return

b. ELSE IF (DATA > TEMP →info)
   i. IF (TEMP →RChild not equal to NULL)
      TEMP = TEMP→RChild
   ii. ELSE
      TEMP→RChild = NewNode
      Return
c. ELSE
   i. Display Duplicate node
   ii. Return

5. Exit

Source Code:

```
void insert(int Data, NODE Root)
{
    NODE NewNode = (NODE)malloc(sizeof(NODE));
    NODE temp;
    NewNode->info = Data;
    if(Root == NULL)
        Root = NewNode;
    else
    {
        temp = Root;
        while(1)
        {
            if(Data < temp->info)
            {
                if(temp->LChild != NULL)
                    temp = temp->LChild;
                else
                {
                    temp->LChild = NewNode;
                    return;
                }
            }
            else if(Data > temp->RChild)
            {
                if(temp->RChild != NULL)
                    temp = temp->RChild;
                else
                {
                    temp->RChild = NewNode;
                    return;
                }
            }
            else
            {
                printf("Duplicate node \n");
                return;
            }
        }
    }
}
```

6.6.3 Algorithm for Searching A Node from BST

1. Input the DATA to be searched and assign the address of the root node to ROOT.
2. TEMP = ROOT
3. Repeat Until DATA is not Equal to TEMP → Info)
   (a) IF (DATA == TEMP → Info)
      i. Display "The DATA exist in the tree"
      ii. Return
   (b) If (TEMP == NULL)
      i. Display "The DATA does not exist"
      ii. Return
   (c) If(DATA > TEMP→Info)
      i. TEMP = TEMP→RChild
   (d) Else If(DATA < TEMP→Info)
      i. TEMP = TEMP→LChild
4. Exit

Source Code:

```
void findBST(int data, NODE Root, NODE *node)
{
    NODE TEMP = Root;
    while(data != TEMP->info)
    {
        if(data == TEMP->info)
        {
            printf("Data Found in tree \n");
            *node = TEMP;
            return;
        }
        if(TEMP == NULL)
        {
            printf("Data Not found in tree");
            return;
        }
        if(data > TEMP->info)
            TEMP = TEMP->RChild;
        else if(data < TEMP->info)
            TEMP = TEMP->LChild;
    }

}
```

6.6.4 Deleting A Node From BST

First search and locate the node to be deleted. Then any one of the following conditions arises:

1. The node to be deleted has no children
2. The node has exactly one child (or sub tress, left or right sub tree)
3. The node has two children (or two sub tress, left and right sub tree)

Suppose the node to be deleted is N.

- If N has no children then simply delete the node and place its parent node by the NULL pointer.
- If N has one child, check whether it is a right or left child.
- If it is a right child, then find the smallest element from the corresponding right sub tree. Then replace the smallest node information with the deleted node.
- If N has a left child, find the largest element from the corresponding left sub tree. Then replace

the largest node information with the deleted node.

The same process is repeated if N has two children, i.e., left and right child. Randomly select a child and find the small/large node and replace it with deleted node. NOTE that the tree that we get after deleting a node should also be a binary search tree.

Lets look an example of deleting a node from BST. Consider the BST of Fig12 and if we want to delete 75 from tree, following steps are obtained.

Step 1: Assign the data to be deleted in DATA and NODE = ROOT.

Step 2: Compare the DATA with ROOT node, i.e., NODE, information of the tree. Since (50 < 75) NODE = NODE → RChild

Step 3: Compare DATA with NODE. Since (75 = 75) searching successful. Now we have located the data to be deleted, and delete the DATA.(Fig28).

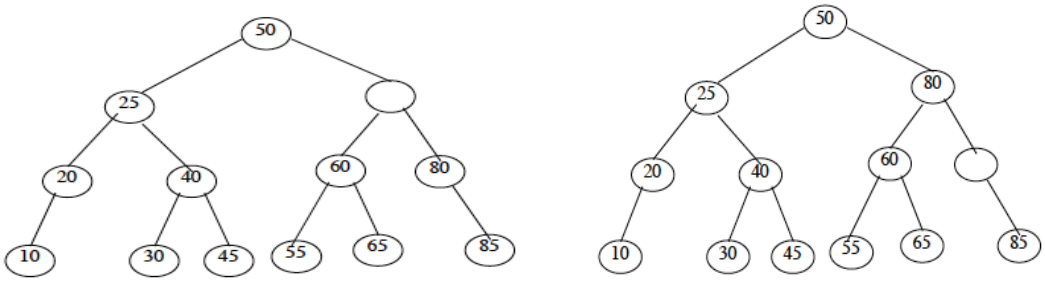


Figure 28: Deletion from BST (a) Delete Node (b) Replace Deleted Node

Step 4: Since NODE (i.e., node where value was 75) has both left and right child choose one. (Say Right Sub Tree) - If right sub tree is opted then we have to find the smallest node. But if left sub tree is opted then we have to find the largest node.

Step 5: Find the smallest element from the right sub tree (i.e., 80) and replace the node with deleted node (Fig 32 (b)).

Step 6: Again the (NODE → RChild is not equal to NULL) find the smallest element from the right sub tree (Which is 85) and replace it with empty node (Fig29).

Step 7: Since (NODE→RChild = NODE→LChild = NULL) delete the NODE and place NULL in the parent node (Fig30).

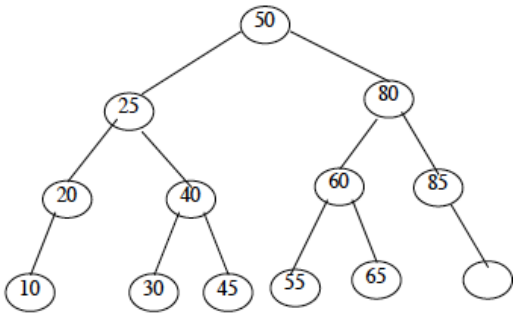


Figure 29: Delete Operation contd..

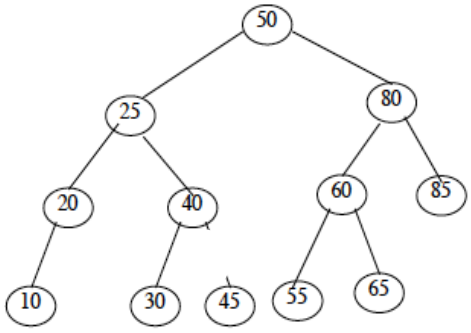


Figure 30: Final Tree after deletion from BST

Step 8: Exit.

6.6.5 Algorithm for deleting a node from BST

NODE is the current position of the tree, which is in under consideration. LOC is the place where node is to be replaced and TEMP is the temporary node. DATA is the information of node to be deleted.

1. Find the location NODE of the DATA to be deleted.
2. If (NODE = NULL)

(a) Display "DATA is not in tree"
(b) Exit

3. If(NODE → LChild == NULL) (Node has no left child)
   (a) NODE = NODE → RChild
4. ELSE If(NODE → RChild == NULL) (Node has no right child)
   (a) NODE = NODE → LChild
5. Else If((NODE → LChild not equal to NULL) && (NODE → RChild not equal to NULL))
   (a) TEMP = NODE →LChild
   (b) LOC = NODE
   (c) While(TEMP → RChild not equal to NULL)
      i. LOC = TEMP
      ii. TEMP = TEMP→RChild
   (d) NODE→info = TEMP→info
   (e) IF(LOC == NODE)
      i. LOC→LChild =TEMP→LChild
   (f) ELSE
      i. LOC → RChild = TEMP→LChild
6. Free TEMP
7. Exit

Source Code:

```
void deleteBST(int data, NODE Root)
{
    NODE node, tmp, loc;
    find(data,Root,&node); // find the node to be delete
    if(node == NULL)
    {
        printf("NO data in Tree\n");
        return;
    }
    if(node->RChild == NULL) //node has no right child
        node=node->LChild;
    else if(node->LChild == NULL) //node has no left child
        node = node->RChild;
    else //node has both child
    {
        tmp = node->LChild;             //tmp = node->RChild;
        loc = node;
        while(tmp->RChild != NULL)      //while(tmp->LChild != NULL)
        {
            loc = tmp;
            tmp = tmp->RChild;          //tmp=tmp->LChild;
        }
        node->info = tmp->info;
        if(loc == node)
            loc->LChild = tmp->LChild; //loc->RChild = tmp->RChild;
        else
            loc->RChild = tmp->LChild; //loc->LChild = tmp->RChild;
    }
    free(tmp);
}
```

6.7 BALANCED BINARY TREE

A balanced binary tree is one in which the **largest path** through the left sub tree is the same length as the **largest path** of the right sub tree, i.e., from root to leaf. Searching time is very less in balanced

binary trees compared to unbalanced binary tree. i.e., balanced trees are used to maximize the efficiency of the operations on the tree. There are two types of balanced trees:

1. Height Balanced Trees
2. Weight Balanced Trees

6.8 Height Balanced Tree

In height balanced trees balancing the height is the important factor. There are two main approaches, which may be used to balance (or decrease) the depth of a binary tree:

a. Insert a number of elements into a binary tree in the usual way, using the algorithm of binary search Tree insertion. After inserting the elements, copy the tree into another binary tree in such a way that the tree is balanced. This method is efficient if the data(s) are continually added to the tree.
b. Another popular algorithm for constructing a height balanced binary tree is the AVL tree.

6.9 Weight Balanced Tree

A weight-balanced tree is a balanced binary tree in which additional weight field is also there. The nodes of a weight-balanced tree contain four fields:

a. Data Element
b. Left Pointer
c. Right Pointer
d. A probability or weight field

- The data element, left and right pointer fields are same as that in any other tree node.
- The probability field is a specially added field for a weight-balanced tree. This field holds the probability of the node being accessed again, that is the number of times the node has been previously searched for.
- When the tree is created, the nodes with the highest probability of access are placed at the top. That is the nodes that are most likely to be accessed have the lowest search time.
- And the tree is balanced if the weights in the right and left sub trees are as equal as possible.
- The average length of search in a weighted tree is equal to the sum of the probability and the depth for every node in the tree.
- The root node contain highest weighted node of the tree or sub tree.
- The left sub tree contains nodes where data values are less than the current root node, and the right sub tree contain the nodes that have data values greater than the current root node.

6.10 AVL TREES

Two Russian Mathematicians, G.M. Adel'son Vel'sky and E.M. Landis developed algorithm in 1962; here the tree is called AVL Tree.

An AVL tree is a binary tree in which the left and right sub tree of any node may differ in height by at most 1, and in which both the sub trees are themselves AVL Trees. Each node in the AVL Tree possesses any one of the following properties:

a. A node is called **left heavy**, if the largest path in its left sub tree is one level larger than the largest path of its right sub tree.
b. A node is called **right heavy**, if the largest path in its right sub tree is one level larger than the largest path of its left sub tree.
c. The node is called **balanced**, if the largest paths in both the right and left sub trees are equal. Fig17 shows some example for AVL trees.

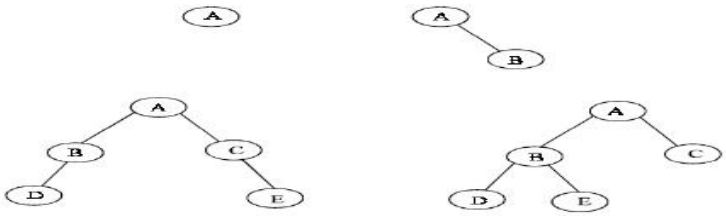


Figure 31: AVL Trees

The construction of an AVL Tree is same as that of an ordinary binary tree except that after the addition of each new node, a check must be made to ensure that the AVL balance conditions have not been violated. If the new node causes an imbalance in the tree, some rearrangement of the tree's nodes must be done.

6.10.1 Algorithm for inserting a node in an AVL Tree

1. Insert the node in the same way as in an ordinary binary tree.
2. Trace a path from the new nodes, back towards the root for checking the height difference of the two sub trees of each node along the way.
3. Consider the node with the imbalance and the two nodes on the layers immediately below.
4. If these three nodes lie in a straight line, apply a single rotation to correct the imbalance.
5. If these three nodes lie in a dogleg pattern (i.e., there is a bend in the path) apply a double rotation to correct the imbalance.
6. Exit.

The above algorithm will be illustrated with an example shown in Fig36, which is an unbalance tree. We have to apply the rotation to the nodes 40, 50 and 60 so that a balance tree is generated. Since the three nodes are lying in a straight line, single rotation is applied to restore the balance.

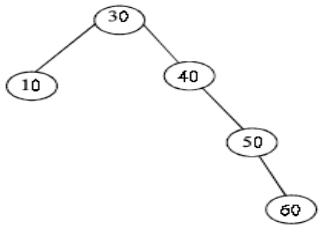


Figure 32: Unbalanced AVL Tree

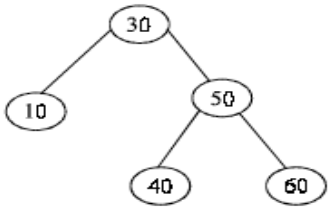


Figure 33: Balanced AVL Tree

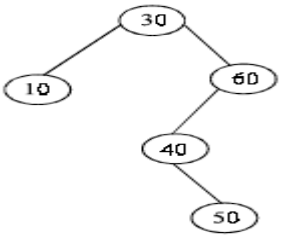


Figure 34: Unbalanced AVL Tree

Fig37 is a balance tree of the unbalanced tree in Fig36. Consider a tree in Fig38 to explain the double rotation.

While tracing the path, first imbalance is detected at node 60. We restrict our attention to this node and the two nodes immediately below it (40 and 50). These three nodes form a ***dogleg pattern***. That is there is bend in the path. Therefore we apply double rotation to correct the balance. A double rotation, as its name implies, consists of two single rotations, which are in opposite direction. The first rotation occurs on the two layers (or levels) below the node where the imbalance is found (i.e., 40 and 50). Rotate the node 50 up by replacing 40, and now 40 become the child of 50 as shown in Fig39.

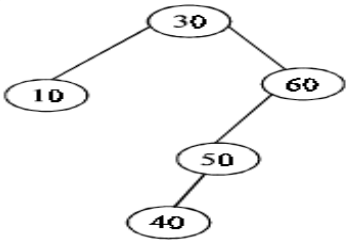


Figure 35: AVL tree after single rotation

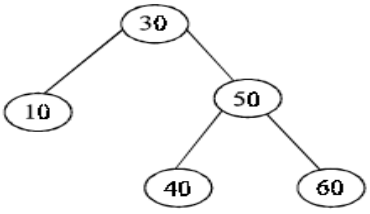


Figure 36: AVL tree after double rotationFig21

Apply the second rotation, which involves the nodes 60, 50 and 40. Since these three nodes are lying in a straight line, apply the single rotation to restore the balance, by replacing 60 by 50 and placing 60 as the right child of 50 as shown in Fig22. Balanced binary tree is a very useful data structure for searching the element with less time.

6.10.2 More Examples on AVL Balanced Tree

An AVL Tree is defined to be a binary search tree with this balance property. In the work that follows, the height of an empty sub tree is defined to be -1.

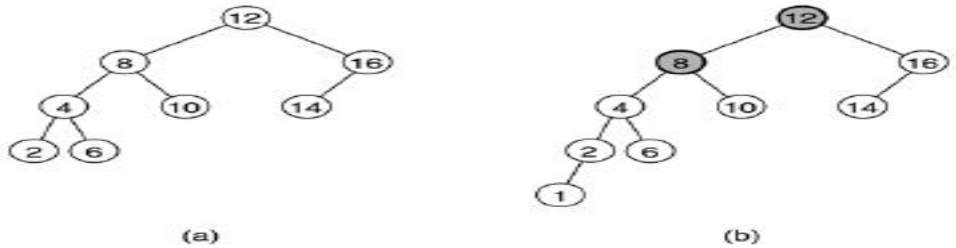
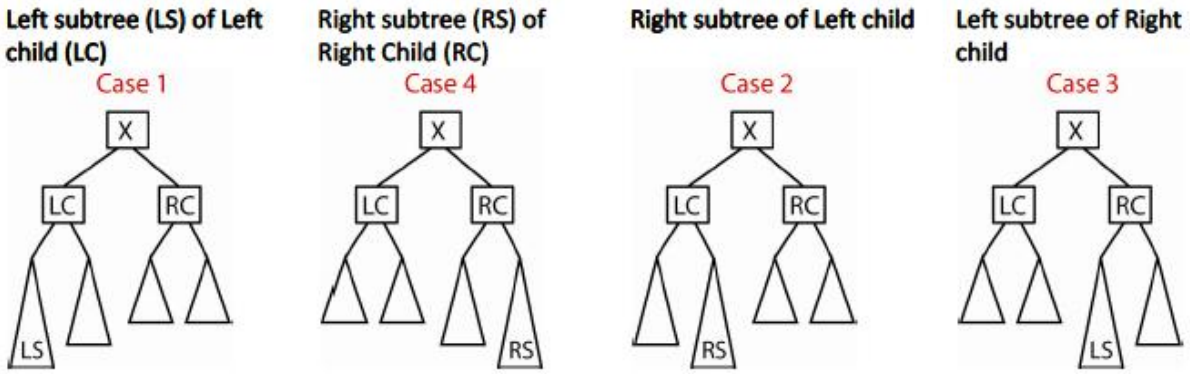


Figure 37: (a) AVL Tree (b) Not AVL Tree

Note in figure 41(a), that insert (1), using the BST algorithm will destroy the AVL property as shown in figure 41(b). Thus, we will have to modify the insert (and remove) algorithms so that they don't destroy the AVL property. The following algorithm, called single rotation, finds the deepest node that violates the property (node 8 in figure 41(b) above) and rebalances the tree from there. It can be proven that this rebalancing guarantees that the entire tree satisfies the AVL property.

Consider inserting a node into an AVL Tree. Suppose a height imbalance of 2 results at some deepest node, X. Thus, X needs to be rebalanced. Assuming an AVL tree (e.g. balance condition met) existed before the insertion, relative to X, we can define these 4 places where the insertion took place:



Case 1: In figure 42 (a) below, node k_2 plays the role of X in preceding figure, the deepest node where the imbalance is observed, and k_1 is the left child of k_2. The idea is to rotate k_1 and k_2 clockwise making k_2 the right sub tree of k_1 and making B the left sub tree of k_2. It is easy to verify that this approach works. First, k_2 is larger than k_1, thus k_2 can be the right child of k_1. Second, all nodes in B are between k_1 and k_2 which is still true when B becomes k_2's left child.

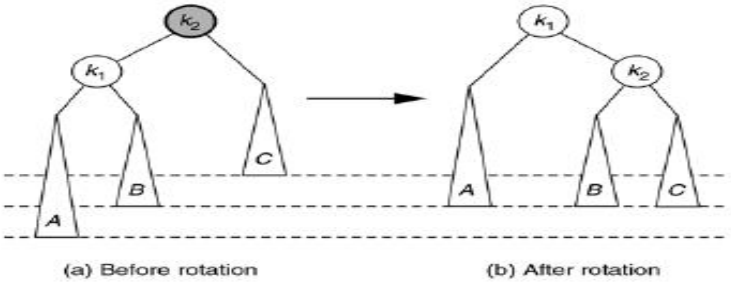
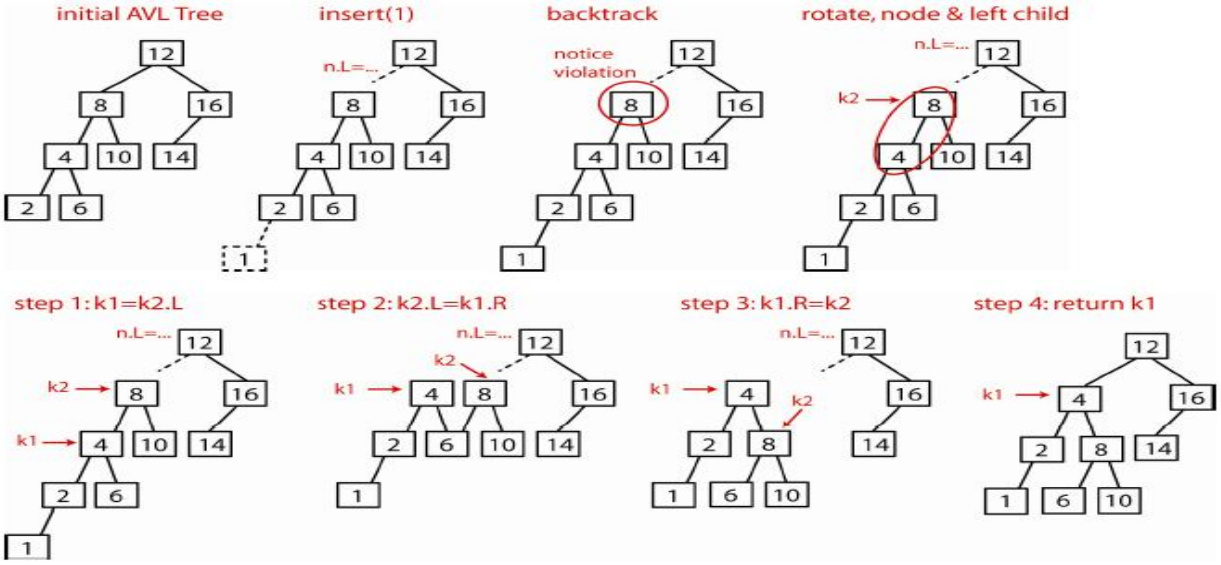


Figure 38: Single rotation to fix case 1

Consider an example of the Case 1 algorithm, which is shown in 4 steps below:



Implementation Algorithm

```
NODE K1, K2;
K1 = K2->left;
K2->Left = K1->Right;
K1->Right = K2;
Return K1;
```

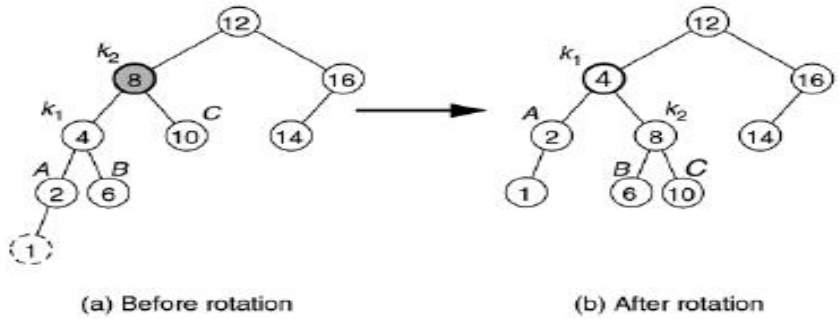


Figure 39: Single rotation fixes an AVL tree after insertion of 1.

Case 4: Consider Case 4, which is a mirror image of Case 1. In Fig 44(b) below, node k_1 plays the role of X, the deepest node where the imbalance is observed, and k_2 is the right child. The idea is to rotate k_1 and k_2 counterclockwise k_1 the left child of k_2 and making B the right child of k_1. It is easy to verify that this approach works. First, k_1 is smaller than k_2, thus it can be the left child of k_2. Second, all nodes in B are between k_1 and k_2 which is still true when B becomes k_1's right child.

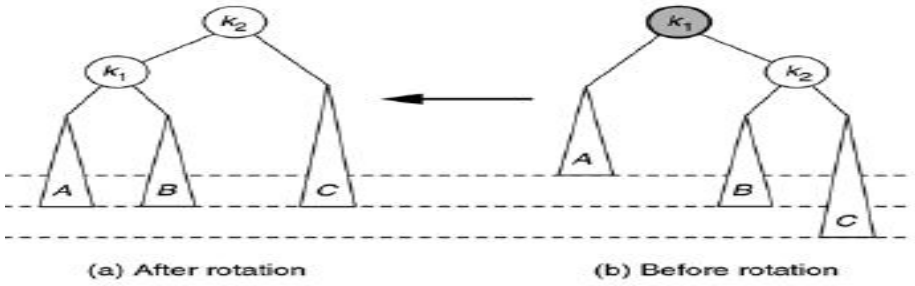
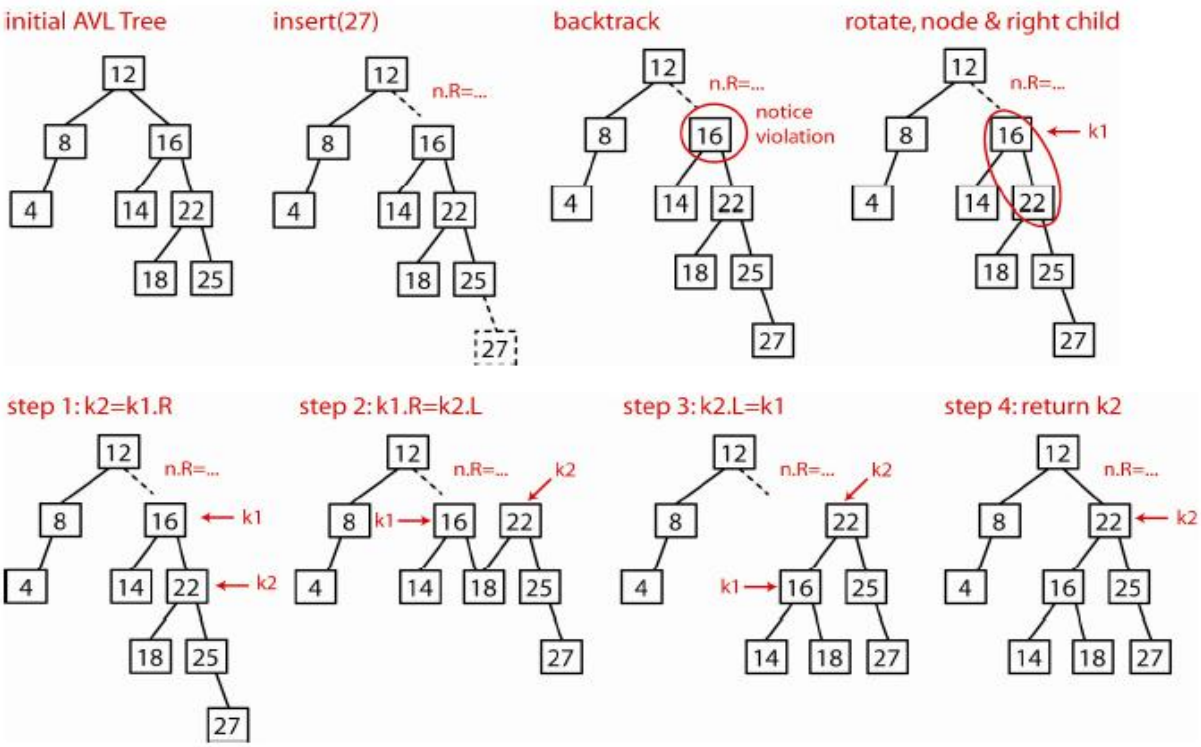


Figure 40: Symmetric single rotation to fix case 4.

Consider an example of the Case 4 algorithm that is shown in 4 steps below:



Implementation Algorithm

```
NODE K1, K2;
K2=K1->right;
K1->right= K2->left;
K2->left = K1;
Return K2;
```

Case 2: Consider Case 2. A rotation as described above, does not work.

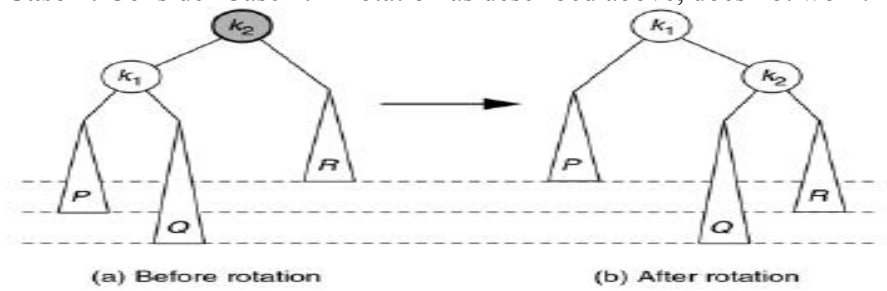


Figure 41: Single Rotation does not fix case2

Consider Case 2 again. A double rotation does work. Since $k_1<k_2<k_3$, the nodes can be rearranged

so that k_1 and k_3 are the left and right, respectively, sub trees of k_2. Since all elements in B are between k_1 and k_2 they remain that way when we make k_1's right child be B. Similarly, since all elements in C are between k_2 and k_3, we can make C k_3's left child.

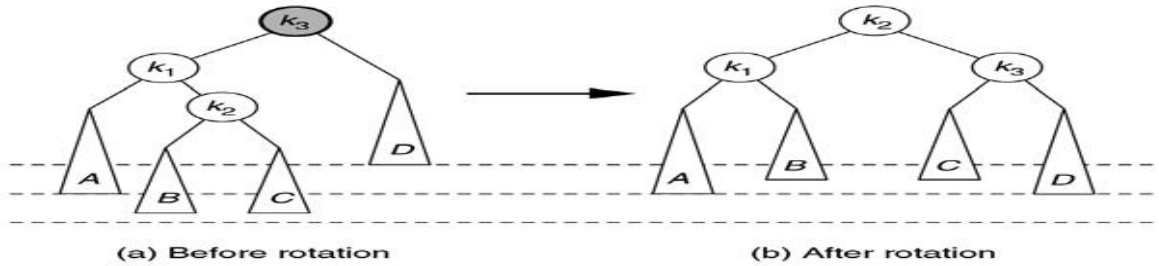
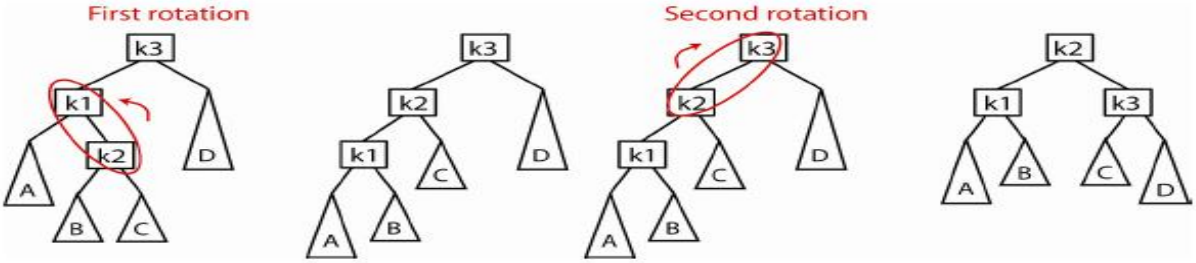


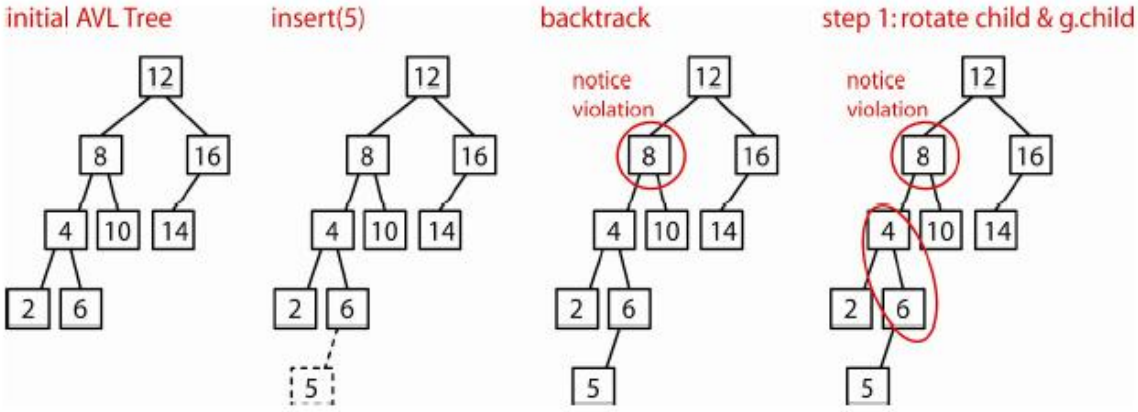
Figure 42: Left-Right Double rotation to fix Case2

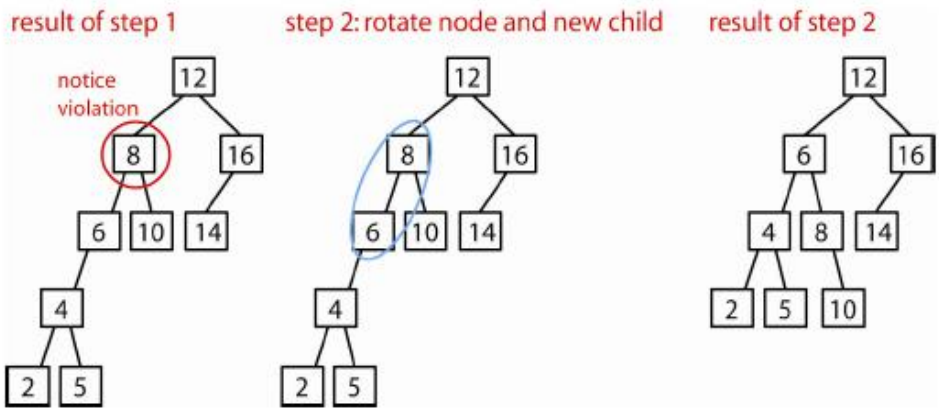
The double rotation can be seen as two single rotations as described for cases 1 and 4:
1. Rotate X's child and grandchild
2. Rotate X and its new child

In figure 27 above, first rotate k_1 and k_2 counter-clockwise, then rotate k_2 and k_3 clockwise:

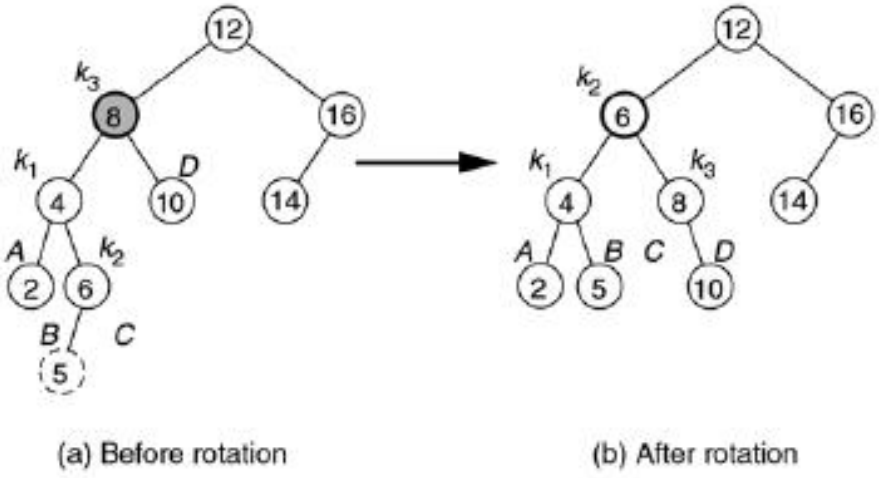


Example





Example



6.11 B-TREE

B-Trees are tree data structure that is most commonly found in database and file system. B-Trees have substantial advantage over alternative implementations when node access times far exceed access times within nodes. This usually occurs when most nodes are in secondary storage such as hard drive. By maximizing the number of child nodes within each internal node, the height of the tree decreases, balancing occurs less often, and efficiency increases.

The idea behind B-Trees is that inner nodes can have a variable number of child nodes within some pre-defined range. Hence, B-trees do not need re-balancing as frequently as other self-balancing binary trees. The lower and upper bounds on the number of child nodes are fixed for a particular implementation.

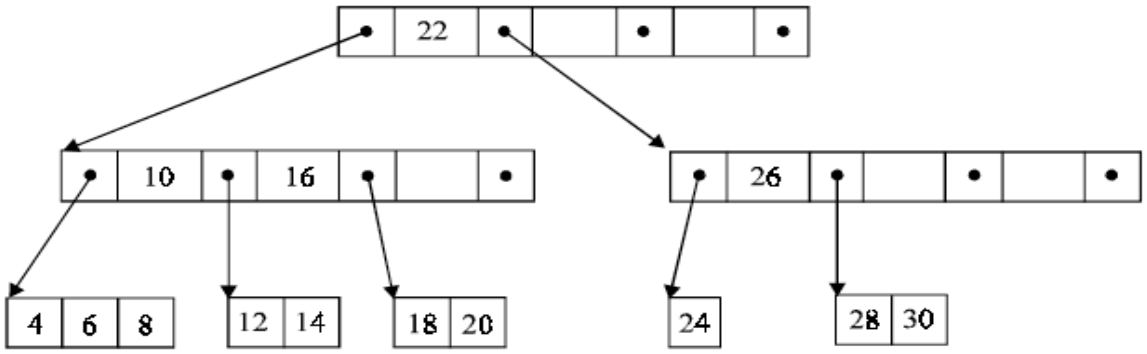


Figure 43: B-Tree

Nodes in a B-Tree are usually represented as an ordered set of elements and child pointers. Every node but the root contains a minimum of m elements, a maximum of n elements, and a maximum of n + 1 child pointers, for some arbitrary m and n. For all internal nodes, the number of child pointers is always one more than the number of elements. Since all leaf nodes are at the same height, nodes do not generally contain a way of determining whether they are leaf or internal.

Each inner node's elements act as separation values which divide its sub trees. For example, if an inner node has three child nodes (or sub trees) then it must have two separation values or elements a_1 and a_2. All values in the leftmost sub tree will be less than a_1,all values in the middle sub tree will be between a_1 and a_2, and all values in the rightmost sub tree will be greater than a_2.

6.11.1 Algorithms in B-Tree

6.11.1.1 Search

Search is performed in the typical manner, analogous to that in a binary search tree. Starting at the root, the tree is traversed top to bottom, choosing the child pointer whose separation values are on either side of the value that is being searched. Binary search is typically used within nodes to determine this location.

6.11.1.2 Insertion

For a node to be in an illegal state, it must contain a number of elements, which is outside of the acceptable range.

1. First, search for the position into which the node should be inserted. Then, insert the value into that node.
2. If no node is in an illegal state then the process is finished.
3. If some node has too many elements, it is split into two nodes, each with the minimum amount of elements. This process continues recursively up the tree until the root is reached. If the root is split, a new root is created. Typically the minimum and maximum number of elements must be selected such that the minimum is no more than one half the maximum in order for this to work.

6.11.1.3 Deletion

1. First, search for the value, which will be deleted. Then, remove the value from the node, which contains it.
2. If no node is in an illegal state then the process is finished.
3. If some node is in an illegal state then there are two possible cases:
   a. Its sibling node, a child of the same parent node, can transfer one or more of its child nodes to the current node and return it to a legal state. If so, after updating the separation values of the parent and the two siblings the process is finished.
   b. Its sibling does not have an extra child because it is on the lower bound. In that case both siblings are merged into a single node and we recurse onto the parent node, since it has had a child node removed. This continues until the current node is in a legal state or the root node is reached, upon which the root's children are merged and the merged node becomes the new root.

6.12 M-WAY SEARCH TREES

Trees having (m–1) keys and m children are called m-way search trees. A binary tree is a 2-way tree. It means that it has m – 1 = 2 – 1 = 1 key (here m = 2) in every node and it can have maximum of two children. A binary tree is also called an m-way tree of order 2. Similarly an m-way tree of order 3 is a tree in which key values could be either 1 or 2 (i.e., inside every node and it can have maximum of two children). For example an m-way tree at order (degree) 4 is shown in following figure.

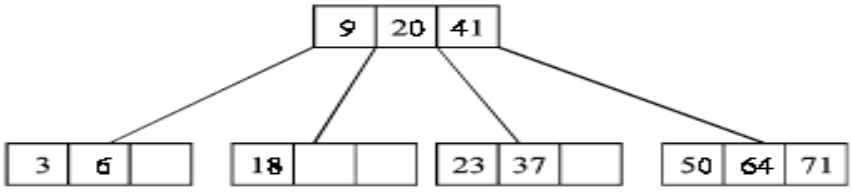


Figure 44: M-way tree of order 4

6.12.1 2-3 TREES

Every node in the 2-3 trees has two or three children. A 2-3 tree is a tree in which leaf nodes are the only nodes that contains data values. All non-leaf nodes contain two values of the sub trees. All the leaf nodes have the same path length from the root node. Fig below shows a typical 2-3 tree.

6.13 HUFFMAN ALGORITHM

- Huffman coding is an entropy encoding algorithm developed by David A. Huffman that is widely used as a lossless data compression technique. The Huffman coding algorithm uses a variable length code table to encode a source character where the variable-length code table is derived on the basis of the estimated probability of occurrence of the source character.
- The key idea behind Huffman algorithm is that it encodes the most common characters using shorter strings of bits than those used for less common source characters.
- The algorithm works by creating a binary tree of nodes that are stored in an array. A node can be either a leaf node or an internal node. Initially, all the nodes in the tree are at the leaf level and store the source character and its frequency of occurrence (also known as weight).
  - Suppose there are 'n' weights w1, w2, --- , wn.
  - Take 2 minimum weights among the 'n' given inputs. Suppose w1, w2 are first two minimum, then the subtree will be
  - Now the remaining weights will be w3, w1 + w2, ---, wn.
  - Create all subtree till the last weight. Thus Huffman algorithms construct the tree from the bottom up approach rather than top down approach.
- **Technique**:

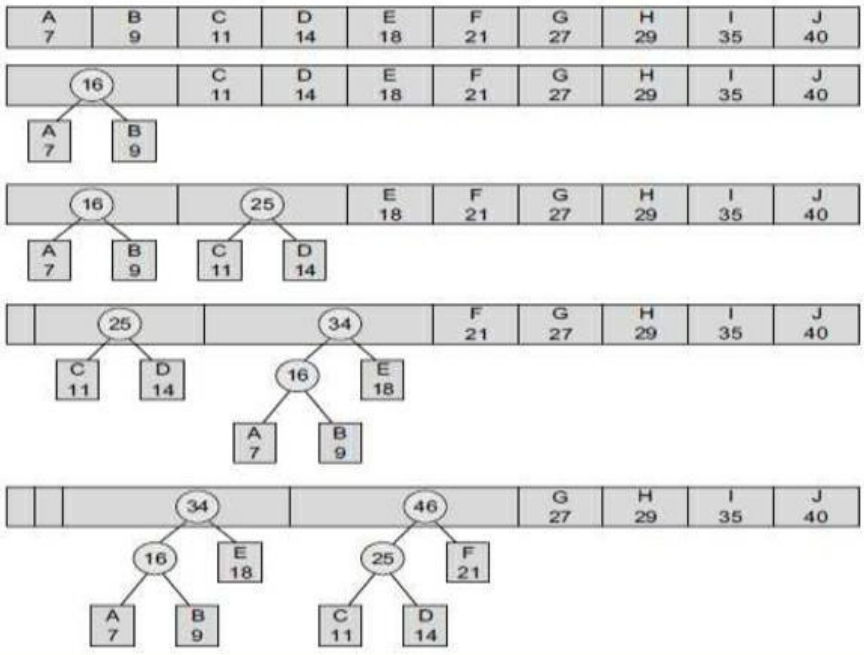
  Given n nodes and their weights, the Huffman algorithm is used to find a tree with a minimum weighted path length. The process essentially begins by creating a new node whose children are the two nodes with the smallest weight, such **that the new node's weight is equal to the sum of the children's weight. That is, the two nodes are merged into one** node. This process is repeated until the tree has only one node. Such a tree with only one node is known as the Huffman tree.

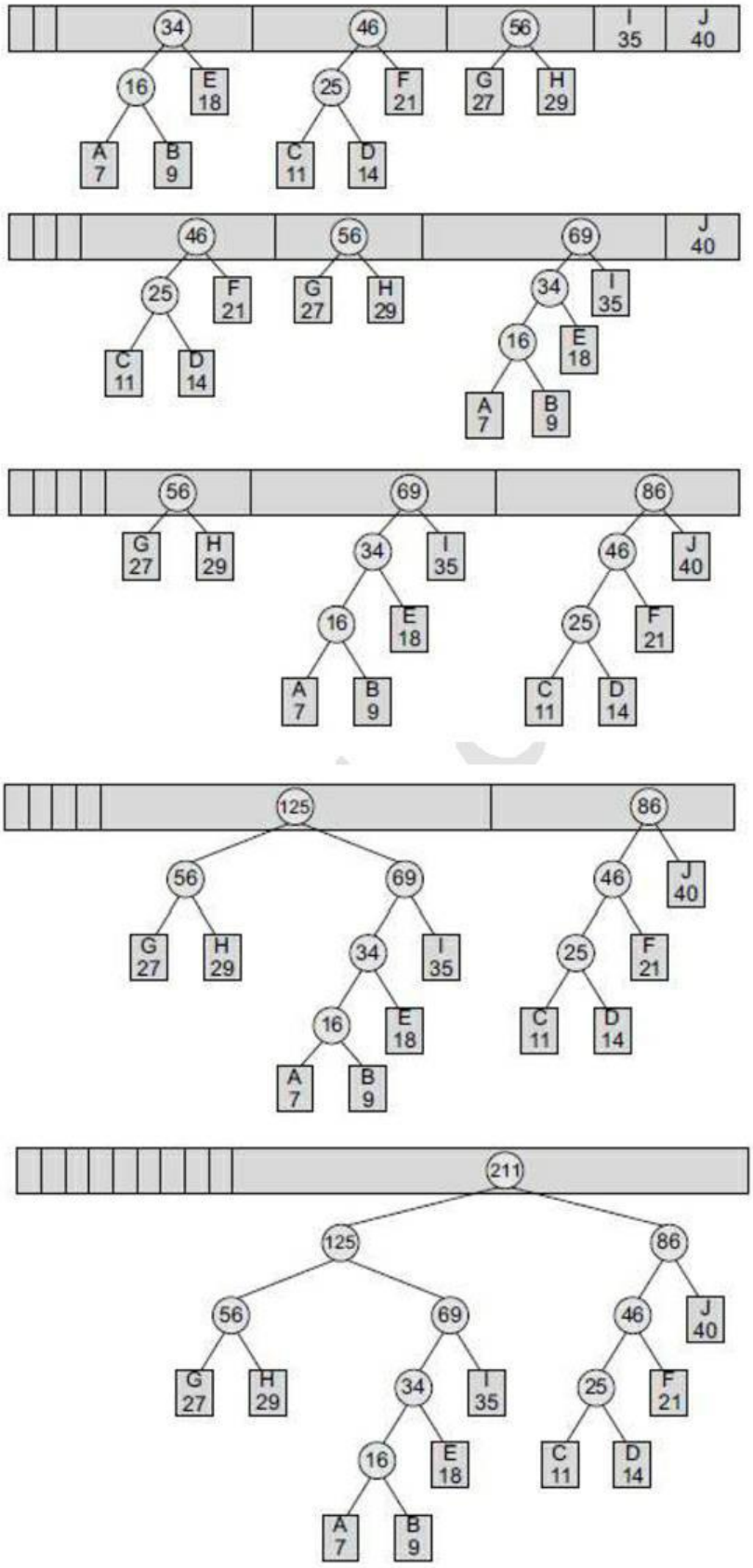
  The Huffman algorithm can be implemented using a priority queue in which all the nodes are placed in such a way that the node with the lowest weight is given the highest priority. The algorithm is shown in Figure A.

```
Step 1: Create a leaf node for each character. Add the character and its weight or frequency
        of occurrence to the priority queue.
Step 2: Repeat Steps 3 to 5 while the total number of nodes in the queue is greater than 1.
Step 3: Remove two nodes that have the lowest weight (or highest priority).
Step 4: Create a new internal node by merging these two nodes as children and with weight
        equal to the sum of the two nodes' weights.
Step 5: Add the newly created node to the queue.
```

Figure A: Huffman algorithm

Example: Create a Huffman tree with the following nodes arranged in a priority queue.




34
46
56
I 35
J 40
16
E 18
25
F 21
G 27
H 29
A 7
B 9
C 11
D 14
46
56
69
J 40
25
F 21
G 27
H 29
34
I 35
16
E 18
C 11
D 14
A 7
B 9
56
69
86
G 27
H 29
34
I 35
46
J 40
16
E 18
25
F 21
A 7
B 9
C 11
D 14
125
86
56
69
46
J 40
G 27
H 29
34
I 35
25
F 21
16
E 18
C 11
D 14
A 7
B 9
211
125
86
56
69
46
J 40
G 27
H 29
34
I 35
25
F 21
16
E 18
C 11
D 14
A 7
B 9

Huffman Coding

In the Huffman tree, circles contain the cumulative weights of their child nodes. Every left branch is coded with 0 and every right branch is coded with 1. So, the characters A, E, R, W, X, Y, and Z are coded as shown in Table

Table Characters with their codes

Character	Code
A	00
E	01
R	11
W	1010
X	1000
Y	1001
Z	1011

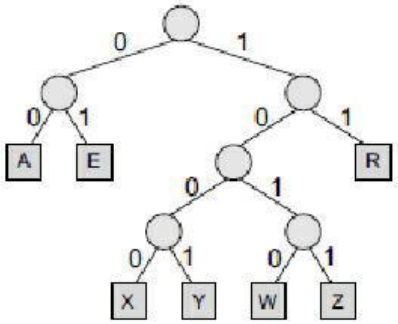


Figure Huffman tree

Thus, we see that frequent characters have a shorter code and infrequent characters have a longer code.

Example:

Huffman's Algorithm with Example

Algorithm: Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \cdots, w_n$

Find a tree T' which gives a solution for the (n-1) weights

$w_1+w_2, w_3, w_4, \cdots, w_n$

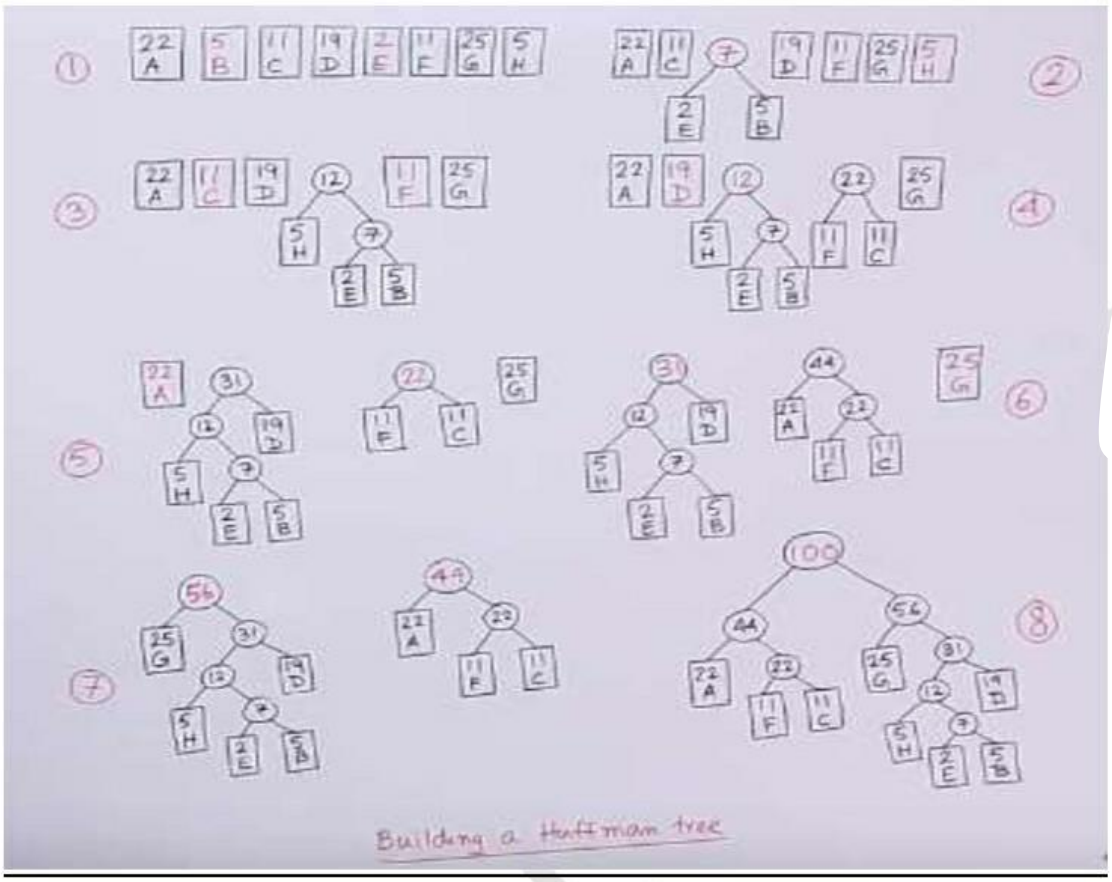
Then, in the tree T', replace the external node w_1+w_2 by the subtree

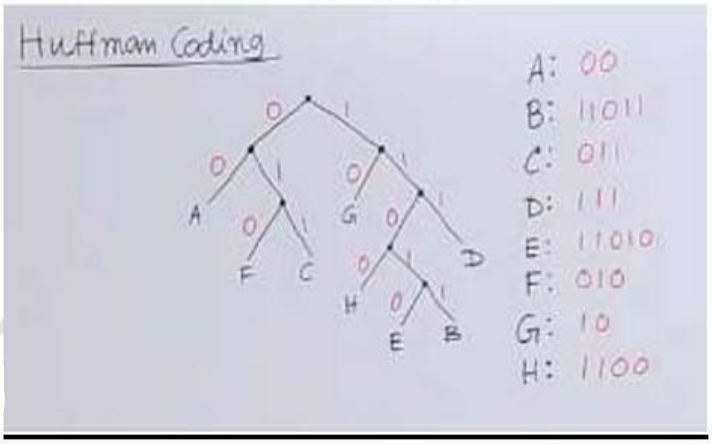
The new 2-tree T is the desired solution

Example: Suppose A, B, C, D, E, F, G and H are 8 data items, and they are assigned weights as follows:

Data items:	A	B	C	D	E	F	G	H
Weight :	22	5	11	19	2	11	25	5

Construct tree T with minimum-weighted path length using the above data and Huffman's algorithm


Building a Huffman tree


Huffman Coding
A: 00
B: 11011
C: 011
D: 111
E: 11010
F: 010
G: 10
H: 1100